

Review: "Research on the Efficiency of Some Deterministic Data Sorting Preprocessing Methods"

V. I. Shynkarenko, O. V. Makarov (2023) · Problems in Programming, No. 4 · Ukrainian State University of Science and Technologies, Dnipro

1. Introduction

The paper "Research on the Efficiency of Some Deterministic Data Sorting Preprocessing Methods" by V. I. Shynkarenko and O. V. Makarov (Problems in Programming, 2023, No. 4) examines whether a deterministic preprocessing pass applied to an array before sorting reduces the total execution time. The authors test the hypothesis that partial ordering of data, performed as a separate stage, accelerates sorting algorithms of differing computational complexity.

2. Methodology

Seven deterministic preprocessing algorithms are proposed. Quick preprocessing estimates the target index of each element by linear interpolation between the minimum and maximum values and performs the corresponding permutation. Preprocessing with memory additionally records occupied positions in a boolean array, preventing repeated predictions for the same index. Reversal preprocessing inverts subsequences ordered in reverse. Block local and block global variants apply the first two algorithms to segments of fixed length. Efficiency is assessed by a disorder measure and by the S indicator of comparative temporal efficiency over 51 trials. The experiments were implemented in C++, with the processor cache cleared before each measurement and the median of repeated runs taken as the result. Quick sort, insertion sort and cocktail sort served as reference algorithms.

3. Results

Preprocessing with memory produced the highest degree of ordering, quick preprocessing reached approximately 62 percent, and reversal preprocessing showed an efficiency close to zero. Combined with insertion sort and cocktail sort, the S indicator approached 1. For Quick sort all variants yielded negative values, since the cost of preprocessing exceeded the gain obtained during sorting. Block local variants consistently outperformed block global ones.

4. Key Insights

Insight 1. The negative result obtained for Quick sort is the most instructive finding. It demonstrates that preprocessing is justified only when the subsequent algorithm is sensitive to the initial ordering of the data. This applies directly to my own work: a recent comparison yielded a variation of 1.5 times for merge sort against more than 4000 times for insertion sort across identical data distributions.

Insight 2. The S indicator is reported in two forms, excluding and including the preprocessing time, and its sign changes between them. Such reporting prevents a partial measurement from

being presented as an overall improvement, and I intend to adopt it when evaluating multi-stage algorithms.

Insight 3. The block length is determined by the processor cache size rather than by any property of the algorithm. This confirms that implementations of equal asymptotic complexity may differ considerably owing to memory access patterns, an effect observed in my own measurements of merge sort.

5. Conclusion

The principal contribution lies not in accelerating sorting as such, but in delimiting the conditions under which preprocessing is expedient. The negative results are as informative as the positive ones, and the accuracy of the experimental procedure makes them credible. A promising direction for further research is the incremental application of preprocessing outlined by the authors, in which the procedure is invoked as new data arrive, maintaining the dataset in a partially ordered state.